

REPORT - CLASSIFICATION

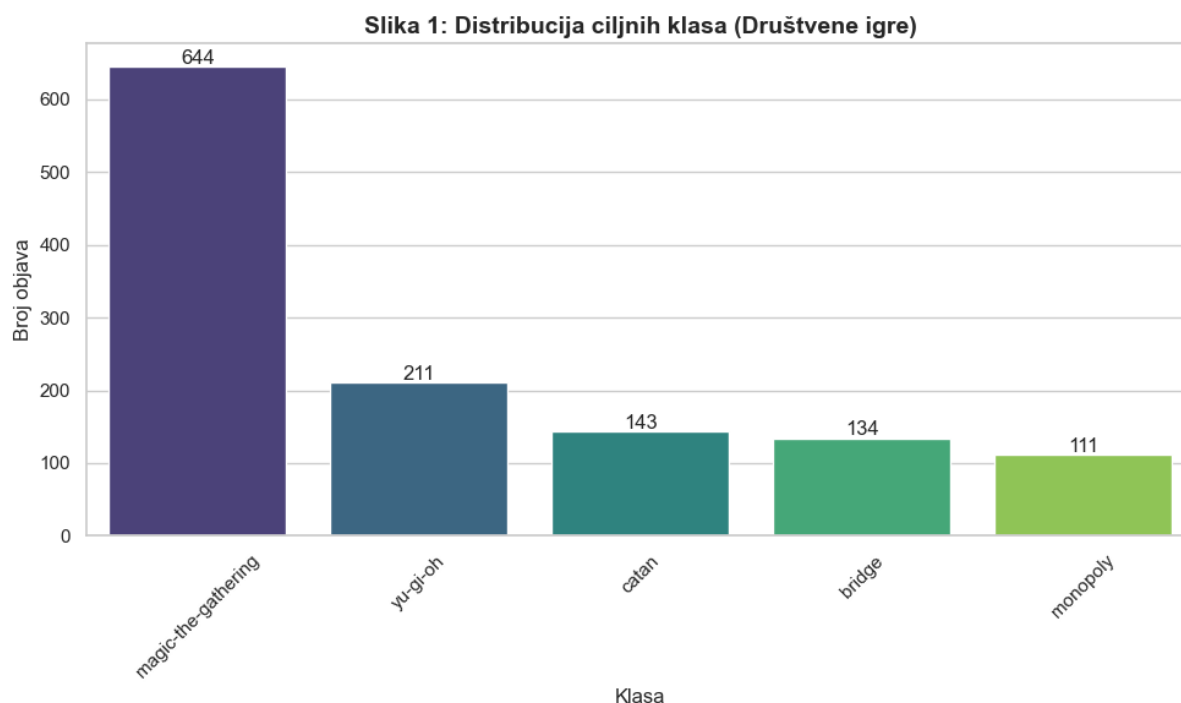
1. Exploratory Data Analysis (EDA)

Work on this classification problem started by loading the training set (train.json) and a detailed inspection of the data structure. The dataset contains textual posts of forum users, accompanied by specific numerical metadata (number of views, score, number of comments), and the goal is to predict which of the five offered board games the given post belongs to.

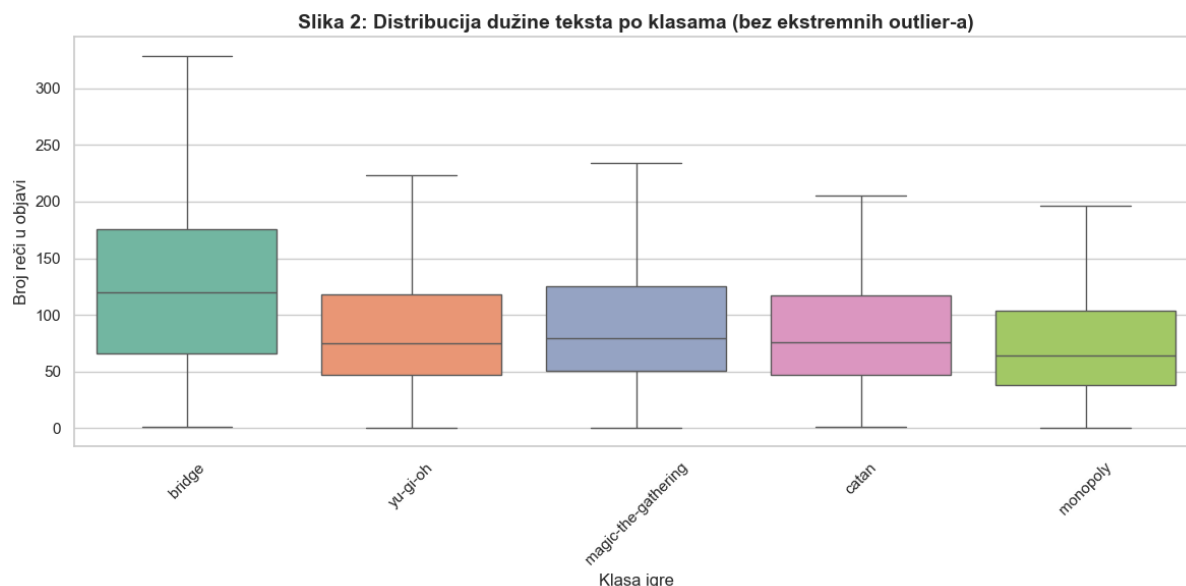
1.1 Overview of Basic Statistics and Visualizations

The first critical point in the analysis was the distribution of the target variable (label). As can be clearly seen in Figure 1, the data suffers from significant class imbalance. The class *magic-the-gathering* dominates the dataset, encompassing more than half of all instances, while the other classes, especially *monopoly*, are significantly underrepresented.

This uneven distribution served as an early indicator that standard classification algorithms will have a tendency to "favor" the majority class, and that it will be necessary to apply weight balancing to achieve the required Macro F1 result.



Furthermore, the length of textual posts (number of words) within each class was examined. Figure 2 shows the distribution of text length after basic HTML tag cleaning. It is noticeable that *magic-the-gathering* and *bridge* forum users tend to provide more detailed descriptions and ask more complex questions, while questions related to *monopoly* are shorter on average.



1.2 Observed Unusual Patterns and Anomalies (Noise)

By detailing examining the raw texts and numerical features, specific, intentionally inserted noises and anomalies were discovered:

1. **Extreme numerical values:** Certain rows had unrealistically negative metrics (e.g., score or views less than -10), as well as abnormally high values in some attributes.
2. **Spam texts:** Posts were observed whose textual content was unusable for modeling. This includes posts with fewer than 5 characters, posts with an excessive proportion of special characters and numbers (where less than 30% of characters are alphabetical), as well as repetitive sequences of the same word. Intentionally mislabeled posts were also noticed, such as a post labeled as *yu-gi-oh* which actually deals with the topic of the euro exchange rate in Vienna.
3. **HTML tags:** The entire text corpus was filled with raw HTML tags (<p>, <a>, href) that would disrupt the vectorization process.

2. Data Preprocessing

Based on the findings from the EDA phase, an exceptionally robust, multi-step preprocessing mechanism was implemented, designed to minimize the impact of observed anomalies before the data enters the training phase.

2.1 Noise Filtering (Heuristics and Anomalies):

The first preprocessing step consisted of discarding rows where the target label was missing. Then, strict logical rules were applied to eliminate negative numerical extremes and evident spam texts. For these purposes, a specialized heuristic function was created *is_spam()*, designed based on observations from the exploratory analysis. After heuristic cleaning, the following was applied: **Isolation Forest** algorithm over log-transformed numerical attributes, detecting and removing more subtle multidimensional outliers. This step is executed exclusively before model training.

2.2 Text Cleaning:

Two separate text cleaning strategies were created, intended for different components of our ensemble model:

1. **std_clean**: Removes all HTML tags, URLs, punctuation, and numbers. The result is clean linguistic text used for standard grammatical and syntactic analysis.
2. **raw_clean**: Removes HTML tags and converts the text to lowercase, but consciously retains punctuation and numbers. This is crucial because forum slang, card names (e.g., "1/1 spirit"), and special characters often carry great semantic value in the context of board games.

2.3 Vectorization – Decision for Character n-grams:

Bearing in mind that the language of the posts is Serbian (a morphologically rich inflected language with cases), standard TF-IDF vectorization at the word level (Word n-grams) observes the words *karta*, *kartu* and *kartama* as three completely independent occurrences. Therefore, the decision was made to apply **character n-gram** vectorization (*analyzer='char_wb'*). By extracting substrings of 2 to 7 characters, the model automatically recognizes the word root (e.g., *kart*), thereby becoming immune to cases and frequent spelling errors of forum users. The function was preserved *sublinear_tf=True* to suppress the excessive influence of words with extreme frequency.

2.4 Temporal Metadata Extraction:

From the variable *creation_date* the year of the post creation was extracted. This is justified by the logic that certain board games had peaks in popularity on forums in different periods (e.g., specific discussion for *Magic the Gathering* in 2013 versus *Catan-a*). This attribute is standardized (StandardScaler).

3. Data Splitting

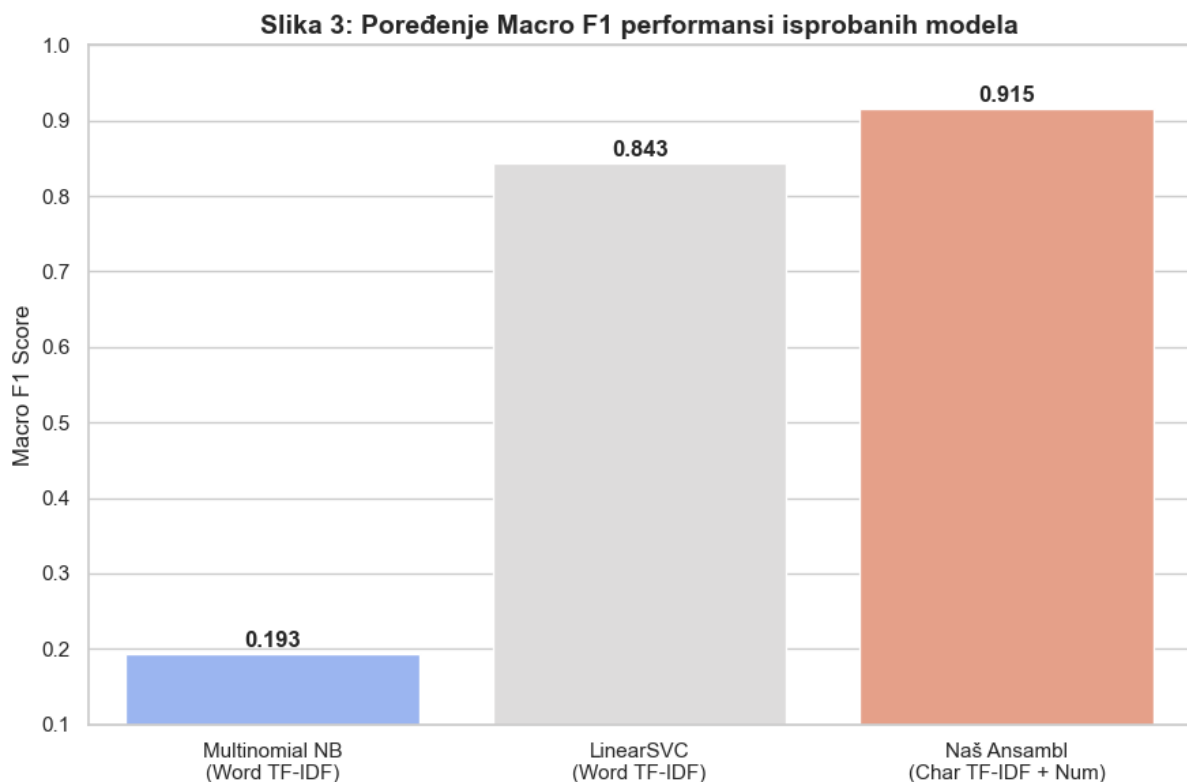
Although the code prepared for the autograder is conceptualized so that, for the sake of achieving maximum generalization power, it trains the model on all available data from the cleaned training set before testing, a crucial part of the validation process took place through the strict method **Stratified Train-Test Split**.

Due to the observed class imbalance from the first phase (Section 1), the data split for testing the algorithms was conducted using the technique *stratify=y_all* (with a ratio of 85% training, 15% validation). This method mathematically guarantees that both the training and validation subsets will contain the exact same percentage of each class (e.g., the class *monopoly* will remain at exactly 9% in both sets). Without such an approach, the estimation of the Macro F1 metric would be unreliable and subject to high variance due to the accidental under-classification of minority classes into the validation set.

4. Tested Algorithms

During the assignment and the search for an optimal solution, three different classification models allowed by the assignment conditions were considered and compared. Their performances are clearly documented in Figure 3.

1. **Multinomial Naive Bayes:** Used as a starting point due to its speed on text data. Although it successfully classifies the majority class, this model has no built-in mechanism for handling class imbalance, which is why it achieved extremely poor results in detecting the class *monopoly*. The Macro F1 measure of this model was below the required threshold.
2. **Linear Support Vector Classifier (LinearSVC):** By introducing `class_weight='balanced'` and using high-dimensional TF-IDF vectors, the SVM algorithm with a linear kernel achieved a significant performance jump. Its decision margin fought the complex text structure more successfully. However, it was rejected in the final solution because it did not provide calibrated results from the function `predict_proba()`, which was essential for creating a more advanced ensemble system.
3. **Logistic Regression Ensemble (Our Model):** The most successful approach proved to be a multiple classifier approach based on Logistic Regression (*LogisticRegression*). Logistic regression with balanced classes proved incredibly efficient when working with sparse matrices created by the character n-gram vectorizer, providing a stable and high *Macro F1 score*.



5. Chosen Solution

The final solution is an advanced **"Soft Voting" Ensemble** system composed of three specialized Logistic Regression models. This approach is designed to exploit the advantages of different preprocessing types and encourages compensating for the shortcomings of individual classifiers.

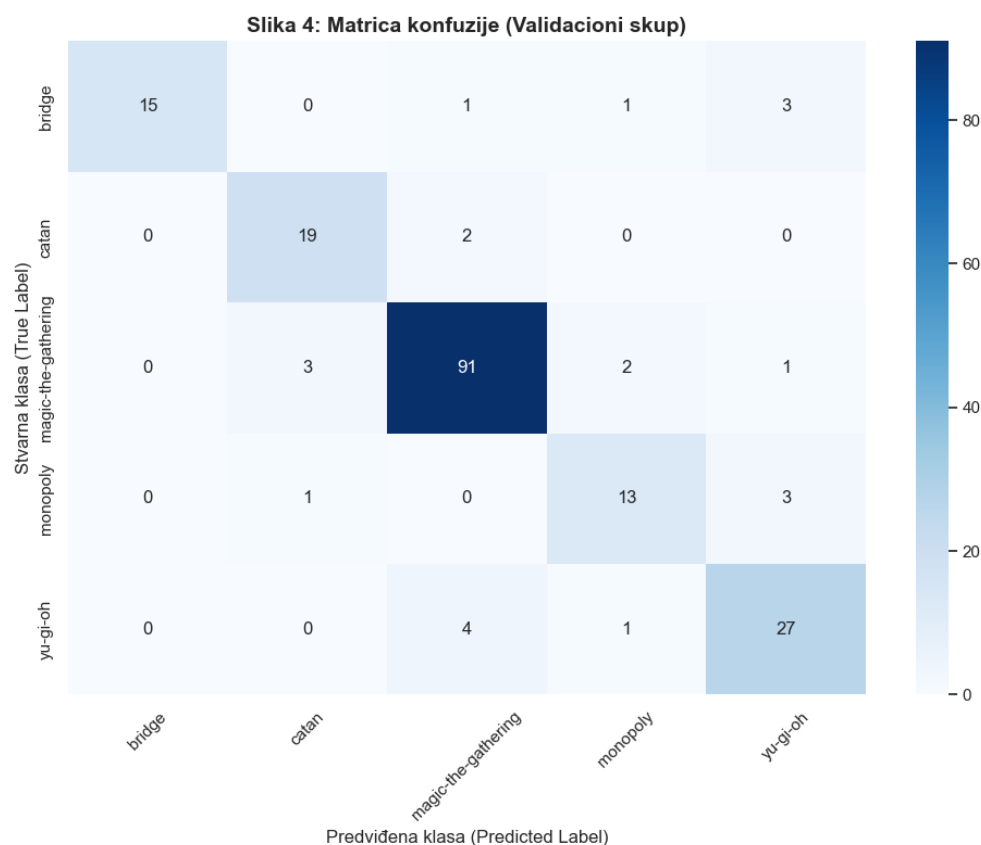
System Architecture:

1. **Grammar model (lr_std):** Trained exclusively on *std_clean* text (only letters, no numbers or punctuation). Its goal is to learn classic linguistic forms.
2. **Raw text model (lr_raw):** Trained on *raw_clean* text (retained characters, punctuation, and numbers). The task of this model is to recognize raw forum slang, tags, and specific board game characters.
3. **Metadata model (lr_num):** Trained exclusively on temporal data (year of post creation) without any insight into the text, making a judgment based on the game's popularity statistics at a given time.

In the classic approach (Hard Voting), the class with the most votes wins, regardless of how confident the models are in their choice. Our system uses the probability recognition method (*predict_proba*). Each of the three mentioned Logistic Regression models returns a confidence level (in percentages) for each of the five games.

These obtained probabilities are then summed according to strict weight factors (*ws*, *wr*, *wn*), which were optimized by an empirical Grid Search approach during the development phase on the validation set. The final decision is made by the function *np.argmax*, choosing the class that has the highest cumulative probability.

Thanks to this voting mechanism and the powerful application of character TF-IDF (*char_wb*), the final model successfully defends against grammatical errors and with incredible precision identifies even the rarest classes (like the class *monopoly*), without the need to excessively raise sensitivity to noise.



6. Use of LLM Tools

During the development of this project, Artificial Intelligence (LLM) tools were used exclusively as assistants to speed up coding and solve specific syntax problems, while the solution architecture, feature engineering, and decision-making were designed independently based on exploratory analysis. Specifically, AI was used for:

- **Regular Expressions Syntax (Regex):** For generating patterns in the functions `is_spam` and `std_clean` (e.g., precise removal of HTML tags and isolation of alphabetical characters).
- **Pandas Manipulations Optimization:** Help with efficiently converting columns to numerical types and solving issues with dates (`creation_date`) within the function for extracting temporal features.
- **Visualization Formatting (Matplotlib/Seaborn):** Generating "boilerplate" code for displaying the confusion matrix and bar charts (adjusting annotations on bars, label rotation, etc.).

7. References

1. Course materials and lecture notes from the Faculty of Technical Sciences, Novi Sad.
2. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing*. (References for N-grams and TF-IDF vectorization in natural language processing).
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*.
4. Scikit-learn Documentation: "Ensemble methods - Voting Classifier" and "Working with Text Data". (Available at: <https://scikit-learn.org/stable/>).
5. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. (Theoretical basis for TF-IDF logic).
6. Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *Eighth IEEE International Conference on Data Mining*.